



中山大學
SUN YAT-SEN UNIVERSITY

本科生实验报告

实验课程: 操作系统原理实验

任课教师: 刘宁

实验题目: 实模式和保护模式下的 OS 启动

专业名称: 计算机科学与技术

学生姓名: 林宏宇

学生学号: 23320093

实验地点: 实验中心 D501

实验时间: 2025 年 03 月 12 日

Section 1 实验概述

学习 x86 汇编基础，学习 x86 汇编、计算机的启动过程、IA-32 处理器架构和字符显存原理，理解实模式下计算机启动的过程，复现“操作系统的启动 Hello World—编写 MBR”部分的实验，并完成课后思考题。

Section 2 预备知识与实验环境

该节总结实验需要用到的基本知识，以及主机型号、代码编译工具、重要三方库的版本号信息等。

- 预备知识：x86 汇编语言程序设计、Linux 系统命令行工具
- 实验环境：
 - 虚拟机版本/处理器型号：Virtualbox
 - 代码编辑环境：Vscode+nasm+C/C++插件
 - 代码编译工具：gcc/g++（64 位）
 - 重要三方库信息：

Section 3 实验任务

- 实验任务 1：复现“操作系统的启动 Hello World—编写 MBR”部分的实验
- 实验任务 2：完成课后思考题 14
- 实验任务 3：完成课后思考题 16
- 实验任务 4：完成课后思考题 17

Section 4 实验步骤与实验结果

----- 实验任务 1 -----

- 任务要求：学习了解 IA-32 处理器知识，学习汇编基础知识，理解实模式下计算机启动的过程，并复现操作系统的启动 Hello World 实验。根据所学的知识，能自己编写程序，然后让计算机在启动后加载运行，以此增进对计算机启动过程的理解，为后面编写操作系统加载程序奠定基础。将学习如何使用 gdb 来调试程序的基本方法。
- 思路分析：

步骤：首先，编写汇编代码并生成相应的二进制（bin）文件。接着，将该文件写入目标存储设备，并使用 QEMU 模拟计算机启动。最后，观察实验结果，

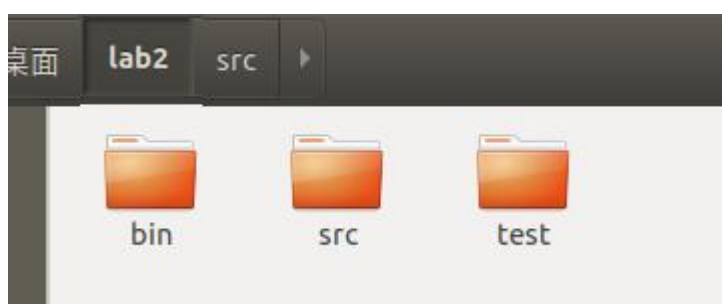
成功打印“Hello, World”即表明实验完成。

注意点：在实验过程中，需要深入理解汇编代码的执行流程，确保代码逻辑正确无误，以避免运行时错误。

● 实验步骤：

1. 编写 MBR(代码见附件), 格式为 mbr.s, 并将代码放在 src 文件下。lab2 文件夹下 bin 用来存放 mbr.bin 文件，test 用来存放调试文件。

文件目录如下：



2. 编译生成 mbr.bin 文件，存放 bin 文件夹下

```
nasm -o bin/mbr.bin src/mbr.s
```

3. 将 mbr.bin 二进制文件装在虚拟机指定的虚拟硬盘，将 mbr 写入 hd.img 的首扇区

```
qemu-img create hd.img 10m  
dd if=./bin/mbr.bin of=hd.img bs=512 count=1 seek=0 conv=notrunc
```

4. 启动 qemu 模拟计算机启动

```
qemu-system-i386 -hda hd.img -serial null -parallel stdio
```

● 实验结果展示：

1. 将 mbr 写入扇区成功，可以观察读入写出记录。

```
linhongyu@linhongyu-VirtualBox:~/桌面/lab2$ dd if=./bin/mbr.bin of=hd.img bs=512  
count=1 seek=0 conv=notrunc  
记录了1+0 的读入  
记录了1+0 的写出  
512 bytes copied, 0.000142415 s, 3.6 MB/s  
linhongyu@linhongyu-VirtualBox:~/桌面/lab2$
```

2. 成功看到输出结果 hello world

```
linhongyu@linhongyu-VirtualBox:~/桌面/lab2$ dd if=./bin/mbr.bin of=hd.img bs=512
count=1 seek=0 conv=notrunc
记录了1+0 的读入
记录了1+0 的写出
512 bytes copied, 0.000142415 s, 3.6 MB/s
linhongyu@linhongyu-VirtualBox:~/桌面/lab2$ qemu-system-i386 -hda hd.img -serial
null -parallel stdio
WARNING: Image format was not specified for 'hd.img' and probing guessed raw.
Automatically detecting the format is dangerous for raw images, write o
perations on block 0 will be restricted.
Specify the 'raw' format explicitly to remove the restrictions.

QEMU
Hello Worldrsion 1.10.2-1ubuntu1)

iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+07F8DD0+07ECDD0 C980

Booting from Hard Disk...
```

----- 实验任务 2 -----

● 任务要求：课后作业第十六题

16.1和16.2使用的都是实模式中断 `int 10h`，由于功能号不同，执行的结果也就不同。在 `int 10h` 中断的资料 https://zh.wikipedia.org/wiki/INT_10H 中，其只给出10h中断下各个功能号的用途，并未给出实际的用法。因此，同学们可能一开始会感觉不知所云，教程下面给出同学们完成本次实验需要用到的功能号。

功能	功能号	参数	返回值
设置光标位置	AH=02H	BH=页码, DH=行, DL=列	无
获取光标位置和形状	AH=03H	BX=页码	AX=0, CH=行扫描开始, CL=行扫描结束, DH=行, DL=列
在当前光标位置写字符和属性	AH=09H	AL=字符, BH=页码, BL=颜色, CX=输出字符的个数	无

注意，“页码”均设置为0。

一般地，中断的调用方式如下。

```
将参数和功能号写入寄存器
int 中断号
从寄存器中取出返回值
```

1. 请探索实模式下的光标中断，利用中断实现光标的位置获取和光标的移动。说说你是怎么做的，并将结果截图。
2. 请修改Hello World的代码，使用实模式下的中断来输出你的学号。说说你是怎么做的，并将结果截图。
3. 在1和2的知识的基础上，探索实模式的键盘中断，利用键盘中断实现键盘输入并回显，可以参考<https://blog.csdn.net/deniece1/article/details/103447413>。关于键盘扫描码，可以参考http://blog.sina.com.cn/s/blog_1511e79950102x2b0.html。说说你是怎么做的，并将结果截图。

● 思路分析：

步骤：首先编写汇编代码，实现预期功能，并生成二进制文件；然后使用工具 `nasm` 进行编译，`qemu` 进行模拟加载并运行程序；最后，观察运行结果，验证实验是否成功。

注意点：需要注意正确使用 BIOS 中断，确保参数设置准确，以避免因调用错误导致程序异常。此外，应仔细检查代码逻辑，确保指令执行顺序符合预期，同时合理设置寄存器，以确保数据正确传递。

● 实验步骤：给出关键部分的代码实现/命令行的执行过程，辅以必要的文字描述（可从指导书上摘抄）。

1. 汇编代码编写 第一小题实现光标位置的获取和移动

实现光标位置获取

```
;获取光标位置并存储光标位置

mov ah, 0x03      ; 功能号 03h: 获取光标位置
mov bh, 0x00      ; 指定页面号 0
int 0x10          ; BIOS 中断, 返回光标的行号和列号

mov [cur_row], dh  ; 存储当前光标的行号
mov [cur_col], dl  ; 存储当前光标的列号
```

实现光标移动

(1) 先根据键盘输入获取信息, 再移动光标位置。

```
cmpal, 'w'
jmove_up
cmpal, 's'
jmove_down
cmpal, 'a'
jmove_left
cmpal, 'd'
jmove_right
jmpkeyboard_loop

;检测输入字符并跳转到相应方向的移动逻辑

move_up:
    decbyte[cur_row]
    jmpupdate_cursor
    ;光标上移

move_down:
    incbyte[cur_row]
    jmpupdate_cursor
    ;光标下移

move_left:
    decbyte[cur_col]
    jmpupdate_cursor
    ;光标左移

move_right:
    incbyte[cur_col]
    ;光标右移
```

(2) 重新输出光标位置, 用 X 显示

```
movah, 0x02
movdh, [cur_row]
movdl, [cur_col]
movbh, 0x00
int0x10
;重新设置光标位置
```

```

movah,0x09

mov al,'X'

movbh,0x00

movbl,0x0F

movcx,1

int0x10

;在新位置显示'X',表示移动成功

```

2. 汇编代码编写 第二小题实模式下输出学号

```

mov ah, 0x01 ; 青色
mov al, '2'
mov [gs:2 * 0], ax
mov al, '3'
mov [gs:2 * 1], ax
mov al, '3'
mov [gs:2 * 2], ax
mov al, '2'
mov [gs:2 * 3], ax
mov al, '0'
mov [gs:2 * 4], ax
mov al, '0'
mov [gs:2 * 5], ax
mov al, '9'
mov [gs:2 * 6], ax
mov al, '3'
mov [gs:2 * 7], ax
mov al, 'L'
mov [gs:2 * 8], ax
mov al, 'H'
mov [gs:2 * 9], ax
mov al, 'Y'
mov [gs:2 * 10], ax

```

3. 汇编代码编写 第三小题 中断实现键盘输入并回显 键盘输入

```

mov ah,0x00;功能号 0: 读取键盘输入字符

int 0x16;调用 BIOS 中断, 等待按键输入

;AL 中返回 ASCII 码字符

```

显示

```

mov ah,0x03;设置颜色

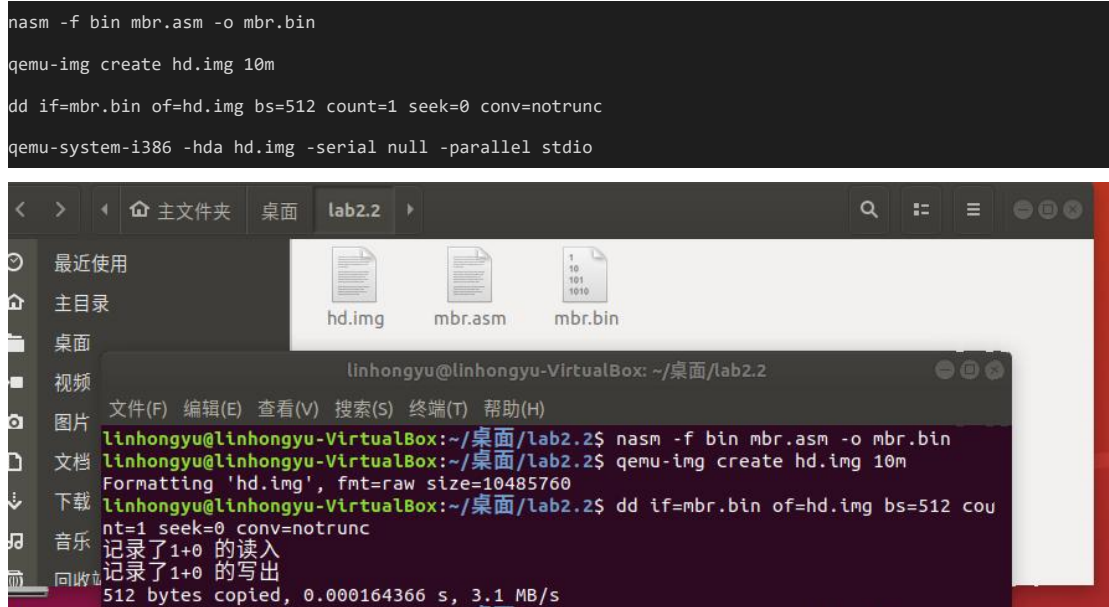
mov [gs:0x0000],al;存储字符

mov [gs:0x0001],ah;存储颜色属性

```

4. 依照同实验任务一相同步骤，执行代码。重点在代码编写部分，后续步

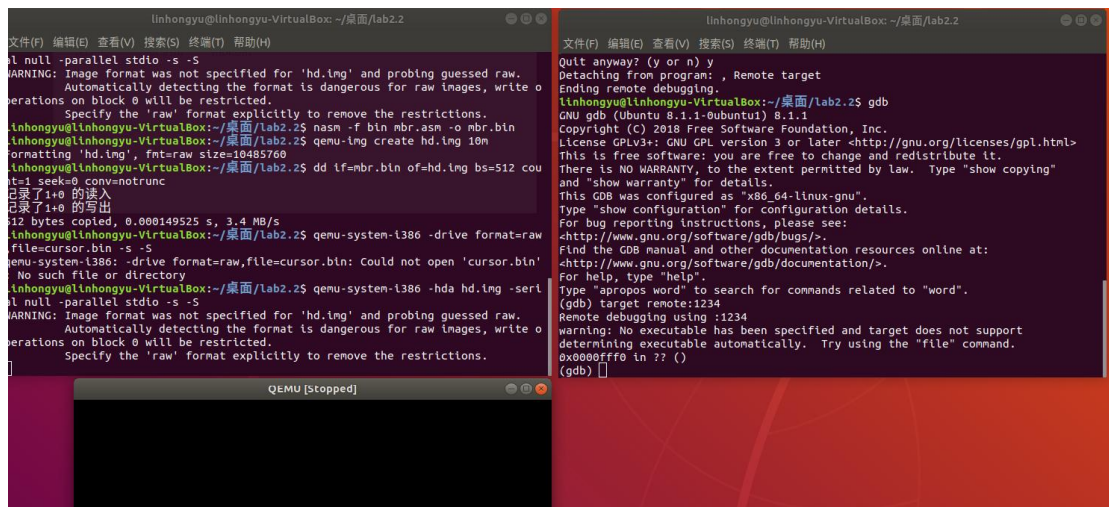
骤和文件存放都如下图相同。



5. 需要进行调试指令，支持终端调试-s-S。

```
qemu-system-i386 -hda hd.img -serial null -parallel stdio -s -S
```

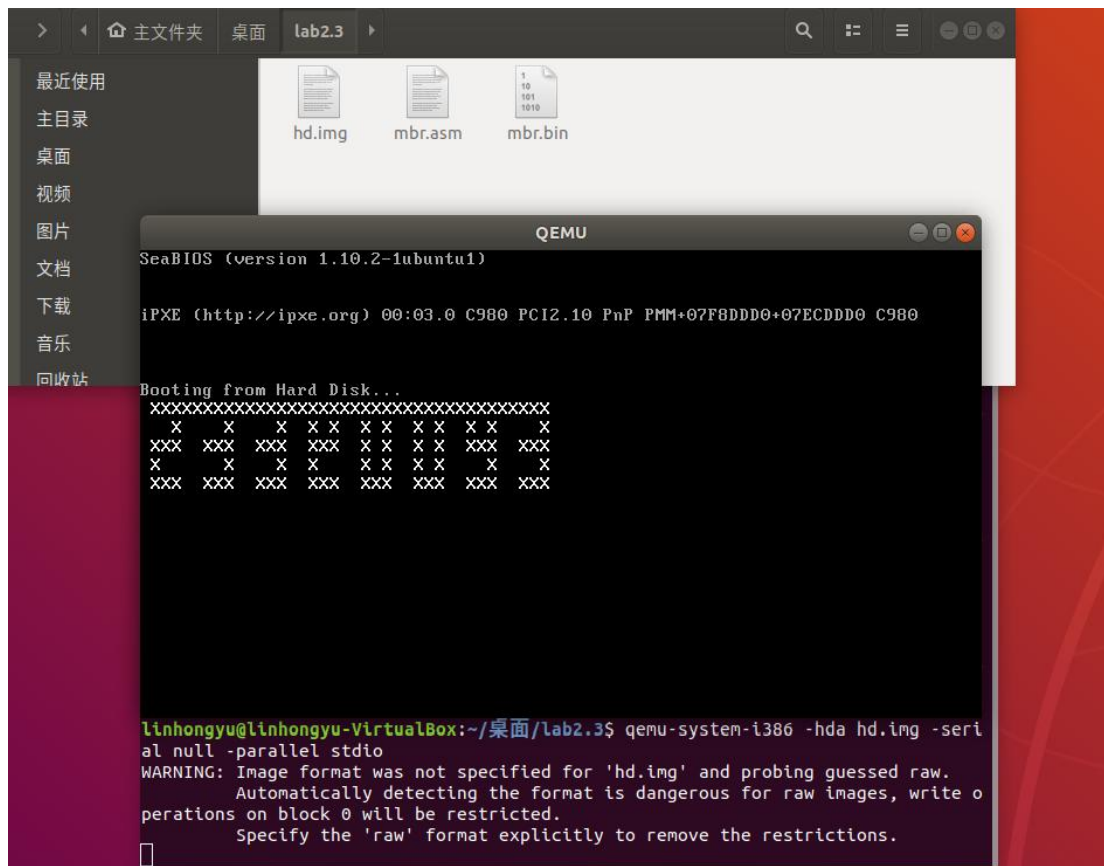
可以 gdb 开启终端进行调试，并设置断点、步进等进行 debug



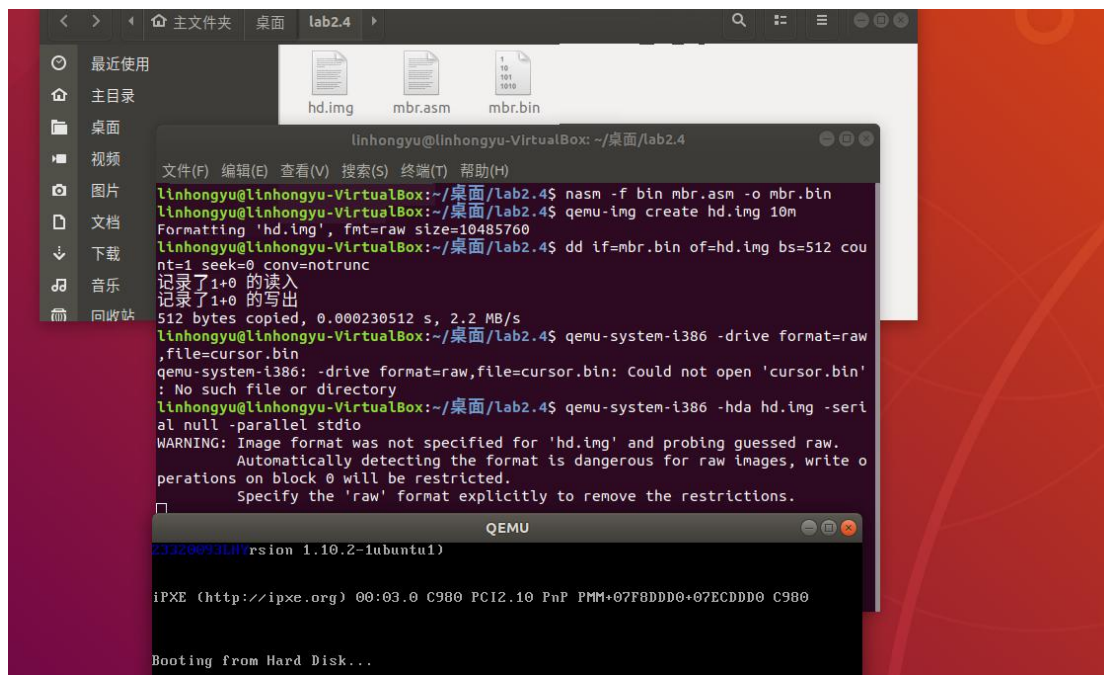
● 实验结果展示：通过执行前述代码，可得下图结果。

1. 第一题获取光标位置和实现光标移动

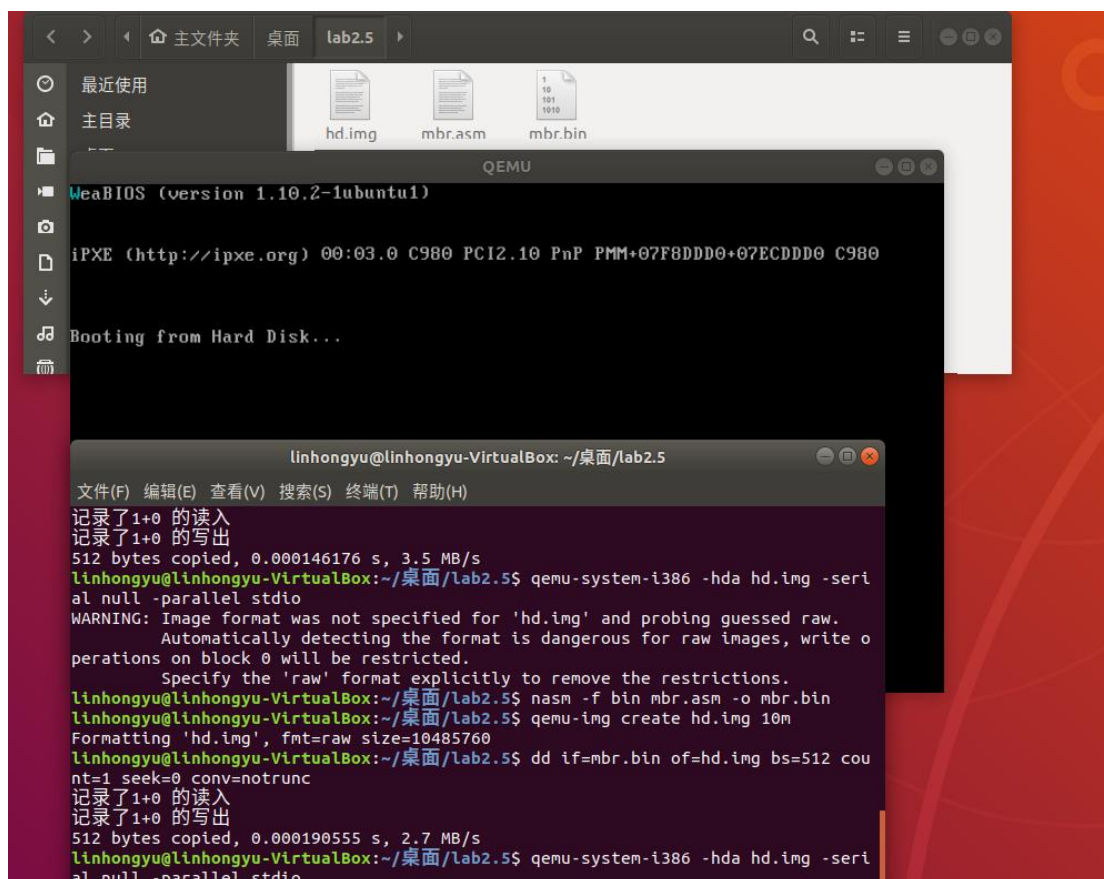
```
linhongyu@linhongyu-VirtualBox: ~/桌面/lab2.2
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
0x0000ffff in ?? ()
(gdb) info registers
eax            0x0          0
ecx            0x0          0
edx            0x663        1635
ebx            0x0          0
esp            0x0          0x0
ebp            0x0          0x0
esi            0x0          0
edi            0x0          0
eip            0xffff0      0xffff0
eflags         0x2          [ ]
cs             0xf000       61440
ss             0x0          0
ds             0x0          0
es             0x0          0
fs             0x0          0
gs             0x0          0
(gdb) p/x dh
没有符号表被读取。请使用 "file" 命令。
(gdb) p/x $dh
$1 = 0x6
(gdb) p/x $dl
$2 = 0x63
```



2. 输出学号



3. 键盘输入和回显



实验任务 3

- 任务要求：实验题十七 汇编代码的编写

17. 汇编代码的编写

- 寄存器请使用32位的寄存器。
- 首先执行命令 `sudo apt install gcc-multilib g++-multilib` 安装相应环境。
- 你需要实现的代码文件在 `assignment/student.asm` 中。
- 编写好代码之后，在目录 `assignment` 下使用命令 `make run` 即可测试，不需要放到 `mbr` 中使用 `qemu` 启动。
- `a1`、`if_flag`、`my_random` 等都是预先定义好的变量和函数，直接使用即可。
- 你可以修改 `test.cpp` 中的 `student_setting` 中的语句来得到你想要的 `a1,a2`。
- 最后附上 `make run` 的截图，并说说你是怎么做的。

● 思路分析：

方法：通过编写汇编代码来实现指定的分支逻辑、循环逻辑和函数。我们需要在环境中安装多架构支持，以便能够编译运行 32 位汇编代码。安装完成后，将伪代码转换为汇编代码。分支逻辑部分要求我们根据变量 `a1` 的值选择不同的操作，并将结果存储到 `if_flag` 中。循环逻辑部分要求根据变量 `a2` 的值执行一定次数的随机数生成，并将每次生成的随机数存储到 `while_flag` 数组中。函数实现部分要求遍历字符串 `string`，并通过调用 `print_a_char` 函数逐个打印字符。完成代码编写后，使用 `make run` 命令进行测试，确保汇编程序按预期执行。

注意事项：调试过程中，需要特别注意寄存器的使用和栈的操作，确保在函数调用时不会改变不应该修改的寄存器。

● 实验步骤：给出关键部分的代码实现/命令行的执行过程，辅以必要的文字描述（可从指导书上摘抄）。

1. 代码编写 `student.asm` 主函数

各个函数代码详细执行看代码文件。

```
student_main:
    ; 初始化 a1 和 a2
    mov dword [a1], 15      ; 设定 a1 的值（可修改）
    mov dword [a2], 16      ; 设定 a2 的值（可修改）

    ; 执行分支逻辑
    call your_if

    ; 执行循环逻辑
    call your_while

    ; 执行遍历字符串函数
    call your_function

    ; 退出程序
    mov eax, 1              ; sys_exit
    xor ebx, ebx            ; 返回码 0
    int 0x80
```

2. 代码编写 test.cpp

```
#include <iostream> //头文件

extern "C" void student_main(); //声明 student.asm 里的 student_main

int a1 = 10; //定义并初始化
int a2 = 15;

void student_setting() {
    a1 = 10;
    a2 = 15;
}

int main() {
    student_setting(); // 设置变量 可以修改 a1 和 a2 的值
    student_main();    // 调用汇编函数
    return 0;
}
```

3. makefile 编写

```
all: run

run: student.o test.o

    g++ -m32 -o test test.o student.o #连接生成的目标文件
    ./test

student.o: student.asm

    nasm -f elf32 student.asm -o student.o #将 asm 文件编译成目标文件

test.o: test.cpp

    g++ -m32 -c test.cpp -o test.o #将 cpp 文件编译成目标文件

clean:

    rm -f *.o test
```

4. 指令运行

```
make clean
make run
```

5. Debug 调试

中途遇到了不少问题，需要检查代码哪里出问题了，选择在 gdb 里步进进行调试，并一步步勘误。

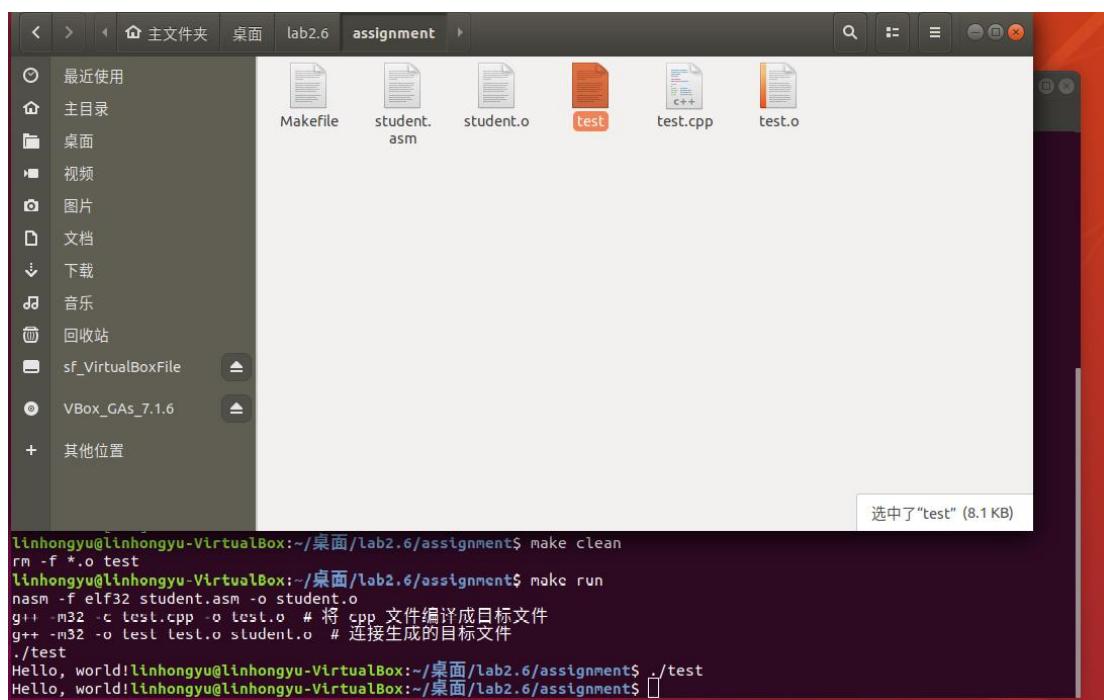
```
gdb
target remote:1234
break student_main
run
next
```

```
(gdb) break student_main
Breakpoint 3 at 0x565558a0
(gdb) run
Starting program: /home/linhongyu/桌面/lab2.6/assignment/test

Breakpoint 3, 0x565558a0 in student_main ()
(gdb) next
Single stepping until exit from function student_main,
which has no line number information.
0x565558b9 in if_case_1 ()
(gdb) next
Single stepping until exit from function if_case_1,
which has no line number information.
0x565558e1 in while_loop ()
(gdb) next
Single stepping until exit from function while_loop,
which has no line number information.
0x5655590c in skip_store_to_while_flag ()
(gdb) next
Single stepping until exit from function skip_store_to_while_flag,
which has no line number information.
0x565558e1 in while_loop ()
(gdb) next
Single stepping until exit from function while_loop,
which has no line number information.
0x5655590c in skip_store_to_while_flag ()
(gdb) next
Single stepping until exit from function skip_store_to_while_flag,
which has no line number information.
0x565558e1 in while_loop ()
(gdb) next
Single stepping until exit from function while_loop,
which has no line number information.
0x5655590c in skip_store_to_while_flag ()
(gdb) next
Single stepping until exit from function skip_store_to_while_flag,
which has no line number information.
0x565558e1 in while_loop ()
(gdb) next
Single stepping until exit from function while_loop,
which has no line number information.
0x5655590c in skip_store_to_while_flag ()
(gdb) next
```

- 实验结果展示：通过执行前述代码，可得下图结果。

观察到运行 ./test 文件后可以看到遍历字符串的输出 Hello.world

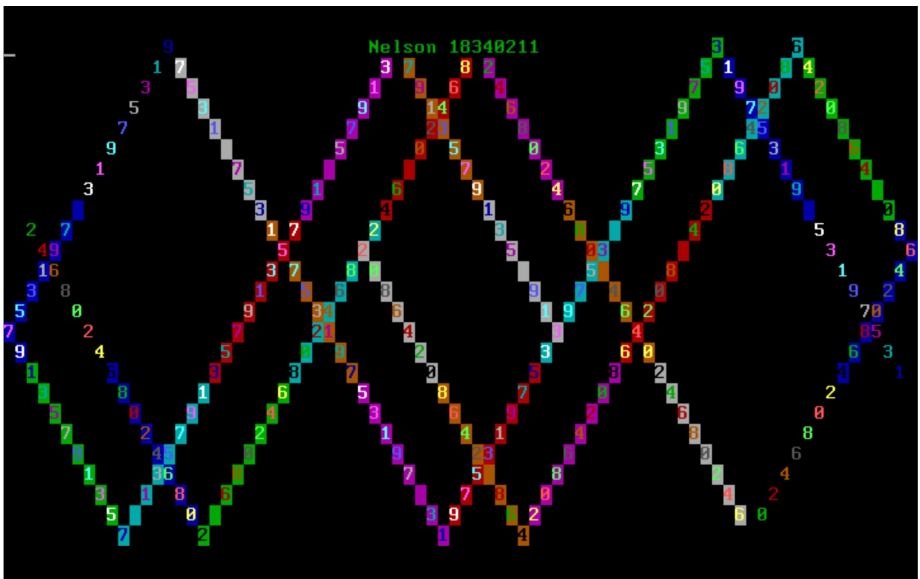


实验任务 4

- 任务要求：课后作业题第十四题

字符弹射程序。请编写一个字符弹射程序，其从点(2,0)处开始向右下角 45 度开始射出，遇到边界反弹，反弹后按 45 度角射出，方向视反弹位置而定。同

时，你可以加入一些其他效果，如变色，双向射出等。注意，你的程序应该不超过 510 字节，否则无法放入 MBR 中被加载执行。静态示例效果如下。



● 思路分析：

步骤：在实现字符弹射程序时，程序的核心逻辑是根据字符与屏幕边界的碰撞情况调整发射方向，同时保证字符在屏幕上持续运动。程序会从初始位置（2, 0）开始，向右下方以 45 度角发射字符，并根据屏幕边缘的碰撞情况反弹。每次反弹后，字符的方向会按照反弹的边界改变，确保按 45 度角的规则继续射出。

注意事项：保证程序能够成功加载并运行在 MBR 中，必须确保整个程序的字节数不超过 510 字节，程序的边界判断要精确无误，避免字符超出屏幕或反弹方向错误。

● 实验步骤：给出关键部分的代码实现/命令行的执行过程，辅以必要的文字描述（可从指导书上摘抄）。

1. 代码编写 展示几个重点部分

显示小球轨迹

```
;画字符（不擦除，留下轨迹）
draw_char:
    mov ah, 0x02    ;设置光标位置
    mov bh, 0       ;页号，0 为当前页面
    mov dh, [y_pos] ;获取当前 Y 坐标
    mov dl, [x_pos] ;获取当前 X 坐标
    int 0x10        ;调用 BIOS 中断，设置光标位置

    mov ah, 0x09    ;写字符
```

```

mov al, [char] ;读取字符

mov bh, 0      ;页号, 0 为当前页面

mov bl, [color] ;读取颜色

mov cx, 1      ;只写一个字符

int 0x10      ;调用 BIOS 中断, 输出字符

ret

```

处理边界

```

;处理边界反弹

cmp byte [x_pos], 79 ;检查是否到达右边界

jg bounce_x_left     ;如果到达, 左反弹

cmp byte [x_pos], 0  ;检查是否到达左边界

jl bounce_x_right    ;如果到达, 右反弹


cmp byte [y_pos], 24 ;检查是否到达下边界

jg bounce_y_up       ;如果到达, 上反弹

cmp byte [y_pos], 0  ;检查是否到达上边界

jl bounce_y_down     ;如果到达, 下反弹

ret

```

更新新坐标

```

;计算新坐标
move_char:

    mov al, [x_pos]

    add al, [x_dir] ;更新 X 坐标

    mov [x_pos], al


    mov al, [y_pos]

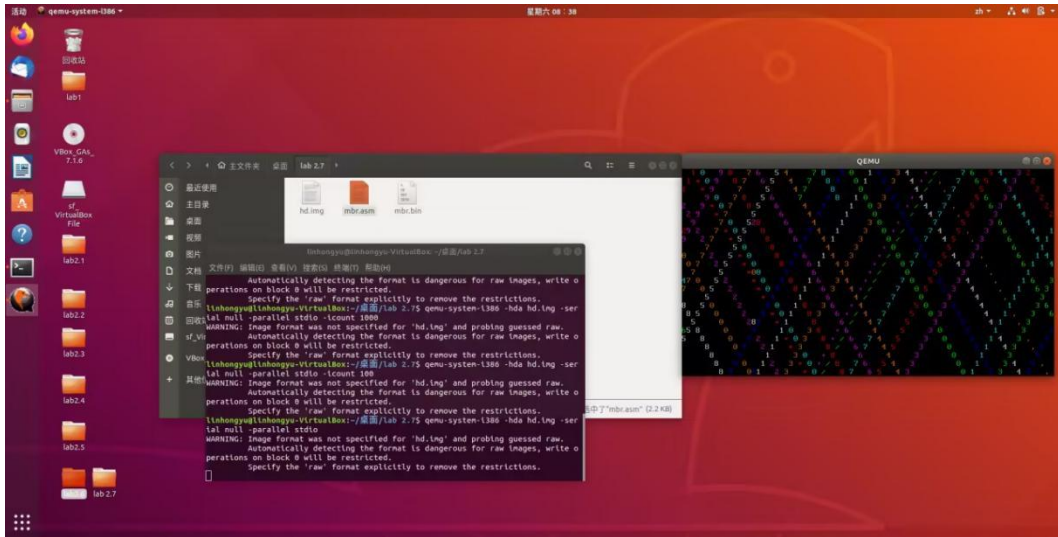
    add al, [y_dir] ;更新 Y 坐标

    mov [y_pos], al

```

2. 按照实验任务一的步骤进行生成和调试, 修改代码, 观察结果。

- 实验结果展示: 通过执行前述代码, 可得下图结果。



Section 5 实验总结与心得体会

1. 通过实验中的第一任务，我进一步加深了对计算机启动过程的理解，特别是在操作系统加载和 MBR（主引导记录）的作用方面。通过编写并成功加载汇编代码到虚拟硬盘，并使用 QEMU 模拟启动，我对计算机如何读取并执行启动代码有了更加清晰的认识。这一过程不仅增强了我的汇编语言能力，还让我明白了计算机启动阶段中各个组件如何协同工作，如何通过二进制文件实现计算机的启动。

2. 调试工具如 GDB 的使用对我帮助巨大。在调试过程中，通过逐步执行代码、设置断点以及观察寄存器的变化，我能够及时发现并解决程序中的问题。这让我更加熟悉了调试流程，并意识到调试对于汇编语言编程的重要性。尤其是在实现光标控制、键盘输入输出等功能时，通过调试定位问题、验证程序正确性，帮助我顺利完成了实验任务。

3. 由于汇编语言的底层性质，每一个细节都可能导致程序的不正常执行。特别是字符反弹和光标移动的逻辑实现需要精确的边界判断和反弹方向的控制。通过仔细分析每一段代码的执行流程和使用条件跳转指令，我逐步解决了这些问题。此外，确保程序符合字节限制（如字符弹射程序不得超过 510 字节）也是一个值得关注的方面，这促使我更加注重代码的精简与优化。

Section 6 对实验的改进建议和意见

无。

Section 7 附录：参考资料清单

参考内容：

https://blog.csdn.net/qq_33339479/article/details/80375085

https://blog.csdn.net/qq_45507768/article/details/131417014

https://blog.csdn.net/zxh2075/category_6459259.html

https://blog.csdn.net/weixin_51756720/article/details/122550393

https://blog.csdn.net/m0_50662680/article/details/131484284

Section 8 附录：代码清单

代码部分在压缩包中。